Proyecto 3

Sistema de Optimización Comparativo para PFSP

Asignatura: Algoritmos Metaheurísticos Inspirados en la Naturaleza
Departamento de Ingeniería Informática — Universidad Católica de la Santísima Concepción (UCSC)

1. Contexto y Evolución del Proyecto

El **Proyecto 3** representa la fase culminante de la serie de proyectos de optimización metaheurística aplicados al agendamiento industrial:

1. **Proyecto 1 (PFSP con AG y AM):** Se modeló e implementó el problema de agendamiento industrial *Permutation Flow Shop Scheduling Problem* (PFSP) utilizando dos enfoques evolutivos de población: **Algoritmos Genéticos (AG)** y **Algoritmos Meméticos (AM)** (AG potenciado con búsqueda local).
2. **Proyecto 2 (PFSP con ACS):** Tras la fase de preentrenamiento con el Problema del Vendedor Viajero (TSP Berlin52), se adaptó e integró el modelo de **Sistema de Colonia de Hormigas (Ant Colony System - ACS)** al problema PFSP, contrastando su desempeño frente a los enfoques evolutivos iniciales.
3. **Preproyecto 3:** Se desarrolló y validó la técnica de **Optimización Extremal (Extremal Optimization - EO / τ-EO)** sobre el Problema de la Mochila 0/1, comprendiendo su dinámica, la evaluación del fitness individual de cada componente (especie) y la selección probabilística por ley de potencia.

Objetivo de esta Etapa Final:
Habiendo generado, probado y validado previamente los tres modelos iniciales (**AG, AM y ACS**), el objetivo del **Proyecto 3** es incorporar el cuarto modelo metaheurístico: **Optimización Extremal (EO)** adaptado a PFSP. Se construirá un **sistema comparativo unificado de cuatro algoritmos (AG, AM, ACS y EO)** que permita evaluar, analizar y discutir de forma rigurosa la eficiencia, escalabilidad y capacidad de exploración/explotación de cada enfoque bajo las mismas instancias de benchmark de Taillard y condiciones experimentales estandarizadas.

2. Definición Formal del Problema (PFSP)

El **Permutation Flow Shop Scheduling Problem (PFSP)** es un problema de optimización combinatoria en el que un conjunto de trabajos debe procesarse en una serie de máquinas en un orden estricto e idéntico.

Componentes:

- n **Trabajos (Jobs):** $J_1, J_2, \ldots, J_n$
- m **Máquinas:** $M_1, M_2, \ldots, M_m$
- **Secuencia idéntica:** Todos los trabajos pasan secuencialmente por las máquinas en el mismo orden: $M_1 \rightarrow M_2 \rightarrow \cdots \rightarrow M_m$.
- **Matriz de Tiempos de Procesamiento ($p_{i,j}$):** Tiempo fijo que requiere el trabajo i en la máquina j.
- **Restricción de Permutación:** El orden de entrada de los trabajos en la máquina 1 se mantiene inalterado en todas las demás máquinas.

Objetivo y Cálculo del Makespan (C_{max}):

Encontrar la permutación de trabajos $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ que **minimice el Makespan (C_{max})**, correspondiente al tiempo en que el último trabajo finaliza su procesamiento en la última máquina.

Para una secuencia π, la matriz de tiempos de finalización $C(i, j)$ (donde i representa la posición en la secuencia $1 \leq i \leq n$, y j la máquina $1 \leq j \leq m$) se calcula de forma recursiva:

$$C(i,j) = \max\Big(C(i-1,j),\ C(i,j-1)\Big) + p_{\pi_i,j}$$

Con las condiciones de borde $C(0,j) = 0$ y $C(i,0) = 0$.

El Makespan de la solución es:

$$C_{max}(\pi) = C(n, m)$$

3. Algoritmos a Implementar y Comparar

El sistema consolidado debe integrar y comparar cuatro metaheurísticas construidas sobre la misma estructura de datos y evaluador de Makespan:

3.1. Algoritmo Genético (AG)

- **Representación:** Cromosoma tipo permutación de longitud n.
- **Selección:** Selección por torneo o ruleta.
- **Cruce:** Operador especializado para permutaciones.
- **Mutación:** Operador de mutación por inserción o intercambio.
- **Reemplazo:** Reemplazo generacional con elitismo.

3.2. Algoritmo Memético (AM)

- **Estructura base:** Algoritmo Genético estándar.
- **Búsqueda Local:** Incorporación de una etapa de explotación aplicada a los mejores individuos de la población (o tras un número fijo de generaciones).
- **Operador de Búsqueda Local:** Inserción local (*insertion local search*) o 2-opt adaptado a permutaciones.

3.3. Ant Colony System (ACS)

- **Representación del Grafo:** Grafo de decisión donde los nodos representan los trabajos y las aristas la transición entre programar el trabajo j inmediatamente después del trabajo i.
- **Matriz de Feromona (τ_{ij}):** Indica la deseabilidad aprendida de programar el trabajo j tras el trabajo i.
- **Información Heurística (η_{ij}):** Visibilidad basada en el tiempo de procesamiento (ej. regla de menor tiempo total de procesamiento SPT o heurística NEH reducida).
- **Regla de Transición Pseudo-aleatoria Proporcional:** Elección del siguiente trabajo equilibrando explotación (con probabilidad q_0) y exploración aleatoria proporcional (con probabilidad $1 - q_0$).
- **Actualización Local de Feromona:** Evaporación local en las aristas recorridas para promover la diversidad de rutas en la misma iteración.
- **Actualización Global de Feromona:** Depósito de feromona exclusivo por parte de la mejor hormiga (global o de la iteración) al finalizar la construcción.

3.4. Extremal Optimization (EO / τ-EO)

- **Inspiración y Dinámica:** Basado en la teoría de auto-organización crítica (modelo de Bak-Sneppen). A diferencia de AG/AM (poblacionales) o ACS (colonia), EO trabaja sobre un único estado/ecosistema local reemplazando sistemáticamente a las componentes con peor nivel de adaptación (*fitness* individual).
- **Representación de Especies:** La permutación actual $\pi = (\pi_1, \ldots, \pi_n)$ actúa como el ecosistema de n especies (trabajos).
- **Fitness Individual (q_i):** Evaluación del nivel de adaptación del trabajo i en su posición actual dentro de la secuencia. Se determina cuantificando el tiempo de inactividad/espera (*idle time*) o el retraso que genera la posición del trabajo i en las distintas máquinas respecto al Makespan parcial.
- **Ranking de Adaptabilidad:** Se ordenan las n componentes según su q_i de menor a mayor adaptabilidad, asignando un ranking $k \in \{1, \ldots, n\}$, donde $k = 1$ corresponde al peor trabajo (menor fitness) y $k = n$ al mejor adaptado.
- **Selección por Ley de Potencia (τ-EO):** Para evitar quedar atrapado en óptimos locales, la componente a modificar no siempre es estrictamente la peor ($k = 1$), sino que se selecciona probabilísticamente utilizando una distribución de potencia:

$$P(k) \propto k^{-\tau}$$

donde $\tau > 0$ es el parámetro de escala que regula la intensidad de la selección.

- **Operador de Mutación / Reemplazo:** La especie seleccionada se modifica aplicando una perturbación local (como mover el trabajo seleccionado a una nueva posición mediante inserción o intercambio con un trabajo aleatorio/vecino).
- **Criterio de Aceptación:** Se acepta incondicionalmente el nuevo estado para mantener la dinámica fuera del equilibrio, registrando de forma persistente la mejor solución global encontrada (π^* y C^*_{max}) a lo largo de las iteraciones.

4. Dataset, Benchmarks y Métricas

Instancias de Prueba:

Se utilizarán las instancias estándar de la literatura de **Taillard**, evaluando al menos tres tamaños de problemas para analizar la escalabilidad:

- **Instancias pequeñas:** $20 \text{ trabajos} \times 5 \text{ máquinas}$.
- **Instancias medianas:** $50 \text{ trabajos} \times 10 \text{ máquinas}$.
- **Instancias grandes:** $100 \text{ trabajos} \times 10 \text{ máquinas}$.

Métrica de Comparación (RPD %):

Dado que cada instancia de Taillard posee un mejor valor conocido (*Upper Bound* / Óptimo), se debe reportar la **Desviación Porcentual Relativa (RPD)** para evaluar la calidad relativa de cada algoritmo:

$$\text{RPD}(\%) = \frac{C_{max}^{\text{obtenido}} - C_{max}^{\text{óptimo}}}{C_{max}^{\text{óptimo}}} \times 100$$

5. Arquitectura del Software

El desarrollo debe mantener la modularidad, claridad y el cumplimiento del principio KISS (*Keep It Simple, Stupid*), separando las responsabilidades en archivos independientes.

Estructura Recomendada de Archivos:

```
proyecto3_pfsp/
│
├── main.py            # Orquestador general y comparación de los 4 modelos
├── operadores.py      # Biblioteca modular de funciones (cálculo C_max, AG, AM, ACS, EO)
├── ag.py              # Ejecutable independiente para Algoritmo Genético
├── am.py              # Ejecutable independiente para Algoritmo Memético
├── acs.py             # Ejecutable independiente para Ant Colony System
├── eo.py              # Ejecutable independiente para Extremal Optimization
├── datos/             # Archivos de instancias de Taillard (.txt)
├── resultados/        # Reportes estructurados de salidas (.csv)
├── graficos/          # Gráficos comparativos de convergencia y boxplots (.pdf / .png)
└── README.md          # Instrucciones de compilación, ejecución y reproducción
```

Ejecución por Línea de Comandos (CLI) y Reproducibilidad:

Cada uno de los cuatro algoritmos debe poder ejecutarse de forma independiente desde la consola recibiendo sus parámetros en orden fijo:

```
python ag.py datos/tai20_5.txt 12345 50 200 0.8 0.1 resultados/res_ag.csv
python am.py datos/tai20_5.txt 12345 50 200 0.8 0.1 10 resultados/res_am.csv
python acs.py datos/tai20_5.txt 12345 50 200 1.0 2.0 0.9 resultados/res_acs.csv
python eo.py datos/tai20_5.txt 12345 5000 1.4 resultados/res_eo.csv
```

El sistema debe garantizar reproducibilidad absoluta mediante el control explícito de la **semilla aleatoria (*seed*)**. Ejecutar cualquiera de los algoritmos con la misma semilla e instancia debe entregar exactamente la misma secuencia final y valor de Makespan.

6. Entregables e Informe Científico

1. Código Fuente (50% de la nota final)

- Implementación modular completa de los cuatro modelos (**AG, AM, ACS y EO**) en **Python 3**.
- Código original, bien estructurado, libre de código redundante e hiper-ingeniería.
- Salida por consola limpia, con formateo claro de avance y manejo básico de errores.

2. Informe Científico (50% de la nota final)

Elaboración de un artículo científico escrito en inglés o español utilizando la **plantilla IEEE Transactions** (extensión sugerida: 2500 a 3500 palabras).

Estructura del Artículo:

1. **Título, Autores y Afiliación.**
2. **Resumen (Abstract):** Síntesis del problema, marco comparativo cuatrilateral y hallazgos principales.
3. **Introducción:** Importancia del PFSP, estado del arte y justificación del uso de metaheurísticas de población (AG/AM), basadas en enjambre (ACS) y basadas en otros elementos de la naturaleza (EO).
4. **Metodología:** Explicación formal de los cuatro algoritmos (**AG, AM, ACS y EO**) aplicados a PFSP, detallando sus representaciones, operadores de variación/transición, funciones de fitness individual/poblacional y parámetros.
5. **Resultados Experimentales:**
   - Tablas comparativas de **Makespan (C_{max})** y **RPD (%)** entre **AG, AM, ACS y EO** para las distintas instancias de Taillard.
   - Comparativa de **Tiempo de CPU** (segundos) consumido por cada modelo.
   - Gráficos comparativos de las **curvas de convergencia** (Makespan vs. Iteración/Evaluaciones) superponiendo los cuatro algoritmos.
   - Análisis crítico multidimensional: tradeoffs entre velocidad de convergencia, calidad de la solución y comportamiento frente a la escala del problema ($n \times m$).
6. **Conclusiones y Trabajo Futuro:** Reflexiones sobre la efectividad de la Optimización Extremal en combinación con búsquedas locales o hibridaciones futuras.
7. **Referencias:** Formato bibliográfico IEEE.

7. Pauta de Evaluación

Criterio	Descripción	Porcentaje
Interfaz y Usabilidad	Presentación por consola, manejo riguroso de argumentos CLI, flexibilidad en scripts y mensajes de error.	**20%**
Calidad de Código y Modularidad	Correcta separación modular en `operadores.py`, cumplimiento del principio KISS, reusabilidad y código original.	**30%**
Funcionalidad y Análisis Comparativo	Correcta implementación de **AG, AM, ACS y EO**, cálculo exacto de C_{max}, reproducibilidad por semillas, rigor experimental y calidad del informe IEEE.	**50%**

Nota sobre Honestidad Académica: El trabajo debe ser 100% original. Cualquier evidencia de plagio o copia resultará en evaluación con nota mínima **NCR** conforme al reglamento institucional.